Final project

Basic recommendation system

Goal

The goal of this assignment is to build a basic movie-to-movie recommendation system.

Throughout the course, we discussed many queries that identified relations between movie pairs and might suggest that people who like the first might also like the second.

Examples include movies of the same director, movies with the same roles, etc.

Note that this is not a course on ML and recommendation systems, though it is a great opportunity to enter the domain.

Teams

You should do the project in pairs. Choose a unique team name. Both team members should submit the same file, using the team name.

Both team members are responsible for the entire project. In case the work is divided between the team members, members should review each other's work.

Implementation

Hint - taking care of data validity problems is critical. Fixing them will boost the performance.

You are not required to implement the system using Python. However, that will allow you to do multiple quick iterations and therefore it is very recommended.

Project subtasks

In order to ease the project, reduce end-of-year stress, and get early feedback,. the project is divided into the following subtasks.

Personal ranking of movies (10%)

The task is done in pairs, twice. One ranks the movies, and the other reviews. Then, switch roles and the other will provide rankings and the other will review.

In this task, we build a collaborative filtering dataset. See last year work as an example.

1. Locally on your server, create the table personal_movies_ranking using the code here. The goal of the table creation is to allow you to run the insert commands and verify their correctness. You should NOT include the table creation in your PR.
2. Each student should rate at least 200 movies based on how much you like them, and provide a justification - see details below. You can choose movies from the following template.
3. You can add other movies too. The movies must be from our IMDB dataset and be identified by their id in the movies files. This is since we should use their meta-data in order to use them for recommendations. Look up the movie that you want to add by its name in the movies table. Add the movie id in the insert statement and write it name in the comment.
4. You should write SQL insert commands inserting the values:
    a. Movie_id - This is the movie id as appearing in the movies table.
    b. Recommendation (see recommendation value semantics)
    c. Suggested_by (e.g., first name, github profile - note the content is public)
    d. Justification - explaining what you like/dislike in the movie. The justification should be specific. Do not just repeat justifications.
    e. Comment - in case that you should write something else. For example, use it if you think that the movie record itself is wrong. Usually this column is empty. .
    f. The template insert statements have a remark with the movie id and its name, to ease understanding. If you add new movies, also add such remarks.
    g. The script should include **only the insert commands**. If all students will recreate the target table, it will end with the content of only the last student.

5. The SQL script should be placed here. The file name should be "personal_ranking_<Your GitHub Profile>.sql"
6. In case that the table protections reject your code, you can find volition using the following script.
7. You should review each other's work to make sure that it contains enough movies, the rating and justification are of high quality, the queries runs, no duplicates, etc.
8. You should submit your rating in a PR. See the course GitHub guide for help with the PR.
9. The reviewer should review the PR. If needed, comments should be given for changes. If all is OK, the reviewer should write Looks Good To Me (LGTM).
10. Submit in the Moodle:
    a. A PDF file named "personal_ranking_<Your GitHub Profile>.pdf"
        i. Your ID and GitHub profile.
        ii. The reviewer ID and GitHub profile.
        iii. The link to the PR in the Moodle.
    b. The sql file from the PR.
11. In case that the PR code will not run the grade will be zero and the reviewer will lose 50 points.
12. Note that we will check the quality of the recommendation and a grade might be reduced due to low quality.
13. The quality of your rating will determine the entire project. Please take care of it seriously.
14. You can choose to avoid using GitHub and just submit the content in the Moodle. In that case your task will be checked only once. If there are problems, the task will be rejected and this section will not increase the project grade.

Movies pairs ranking (10%)

1. Submit at least 200 pairs of movies, with recommendation level (1 - bad recommendation, 5 - neutral, 10 -great), and a justification. 100 pairs should be good recommendations, 100 should be not good yet reasonable recommendations (e.g., the bad movie in a series). See last year's work as an example.
2. Create on your local MySql the table movies_recommendations
3. Create your recommendations file in this directory. The file name should be "movie_pairs_ranking_<Your GitHub Profile>.sql"
4. Submit the recommendations as a PR. The recommendations of the class will serve as the ground truth.
    a. Base_movie_id (people the like this movie)

    b. recommended_movie_id (also like/dislike this movie)
    c. Recommendation (10 - like a lot, 1- don't like)
    d. Suggested_by (e.g., first name, github profile - note the content is public)
    e. Justification - explaining the recommendation value. The justification should be specific. Do not just repeat justifications.
    f. Comment - in case that you should write something else.
    g. The template insert statements have a remark with the movie id and its name, to ease understanding. If you add new movies, also add such remarks.
5. Please focus on pairs of interests. Rank by at least one of the following option or suggest and justify a new method (a good new method will get a bonus).
    a. Rank false positives of interest. Choose a similarity model (e.g., common roles), Run the logic on the movies that you personally ranked. Interesting false positives are movies that are similar yet you ranked one as high and one as low. Explain the matching logic that you choose, justify it, and report the number of pairs found.
    b. Rank false negatives of interest. Choose a similarity model, with high sensitivity (e.g., common roles). Look at the cartesian product of the movies that you personally ranked and like, that are on the threshold (e.g., exactly common roles, no common roles). Explain the matching logic that you choose, justify it, and report the number of pairs found.
    c. Ranking pairs of movies that are recommended to you based on your movies ranking from the previous task.
    d. Complete with pairs of your choice if needed. Avoid null records, and in the "not good yet reasonable" recommendation", provide pairs that have some relation yet not a good recommendation.
6. Justification to moderate recommendation should be like "**Though both movies are A, it is a bad recommendation since B"**
7. You should review each other's work to make sure that it contains enough movies, the rating and justification are of high quality, the queries runs, no duplicates, etc.
8. You should submit your rating in a PR. See the course GitHub guide for help with the PR.
9. The reviewer should review the PR. If needed, comments should be given for changes. If all is OK, the reviewer should write Looks Good To Me (LGTM).
10. Submit in the Moodle:
    a. A PDF file named "movie_pairs_ranking_<Your GitHub Profile>.pdf"
        i. Your ID and GitHub profile.
        ii. The reviewer ID and GitHub profile.
        iii. The link to the PR in the Moodle.
    b. The sql file from the PR.

11.
12. In case that the PR code will not run the grade will be zero and the reviewer will lose 50 points.
13. Note that we will check the quality of the recommendation and a grade might be reduced due to low quality.
14. The quality of your rating will determine the entire project. Please take care of it seriously.
15. You can choose to avoid using GitHub and just submit the content in the Moodle. In that case your task will be checked only once. If there are problems, the task will be rejected and this section will not increase the project grade.

Models plan (30%)

1. Choose a performance metric (e.g., precision, recall) that you would like to optimize (lighthouse metric) and why you choose to optimize it.See example of recall oriented models.
2. Describe 5 models defining a relation of similarity between two movies.
3. You are allowed and encouraged to enhance the database with external data that will improve the models. For example, given the Wikipedia list of actors you can identify the main one. Movies that share a star are more likely to be similar than movies that share only an extra.
4. For each model
    a. Explain the intuition behind the model
    b. Explain the parameters (e.g., number of common actors).
    c. Explain how do you plan to choose the value of the parameters that serve your lighthouse metric best
    d. Explain data quality problems that might hurt the performance and how do you plan to improve them.
5. Explain how you would like to aggregate the predictions of the models into a single one.

Final project submission (50%)

1. Implement your plan.

2. Submit files with the code sql/python and another file with the explanations.
3. Evaluate the performance of **each of your models** and the **aggregated results**. The performance evaluation should include at least precision, recall and your lighthouse metric.
4. Explain the performance results. Justify by numbers.
5. Explain your data quality layer contribution. Justify by numbers.
6. See the project checking guideline to verify that you do not lose points needlessly or miss opportunities for bonuses.